Robotics and Autonomous Systems 196 (2026) 105248

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

Agentic skill discovery☆

Xufeng Zhao *, Cornelius Weber, Stefan Wermter

Department of Informatics, University of Hamburg, Vogt-Koelln-Str. 30, Hamburg, 22527, Germany

ARTICLE INFO

Dataset link: https://github.com/xf-zhao/Agentic-Skill-Discovery

Keywords:
Language models
Robot learning
Autonomous robots
Reinforcement learning
Skill discovery
Environment exploration

ABSTRACT

Language-conditioned robotic skills make it possible to develop the high-level reasoning of Large Language Models (LLMs) for low-level robotic control. A remaining challenge is to acquire a diverse set of fundamental skills. Existing approaches either manually decompose a complex task into primary robotic actions in a top-down fashion, or bootstrap as many combinations as possible in a bottom-up fashion to cover a wider range of task possibilities. These decompositions or combinations, however, require an initial skill library. For example, a "grasping" capability can never emerge from a skill library containing only diverse "pushing" skills. Existing skill discovery techniques with reinforcement learning acquire skills by exhaustive exploration but often yield non-meaningful behaviors. In this study, we introduce a novel learning framework for autonomous robot skill discovery that is entirely driven by LLMs. The framework begins with an LLM generating task proposals based on the provided scene description and the robot's configurations, aiming to incrementally acquire new skills upon task completion. For each proposed task, a series of reinforcement learning processes is initiated, utilizing reward and success determination functions sampled by the LLM to develop the corresponding policy. The reliability and trustworthiness of learned behaviors are ensured by an independent vision-language model. We show that starting with zero skill, the skill library emerges and expands to more and more meaningful and reliable skills, enabling the robot to further propose and complete advanced tasks efficiently.

1. Introduction

"In the beginning was the Word" [1]. Can Large Language Models (LLMs) unleash the generative potential of words, as implied in this famous quote? In a more practical setting, can an LLM teach the skills and goals of human actions to a robot that is initialized without any skills or motivation?

Large Language Models show potential in many fields that require common sense and reasoning. Large-scale models excel because of their training on human datasets, and textual or even multimodal reasoning capabilities. LLM-based agents, especially robots, extend the potential to embodiments, but they still show limitations when applied to direct robotic control. The reasons are insufficient real-world robot data for training, as well as the diversity of topologies and physical properties. As a workaround, abstracting robot control to a certain level and referring to each abstraction as a specific "skill" helps LLMs to control robots generically [2–7]. For example, SayCan [2] builds a robot that can follow a set of basic language instructions. When commanded with a complex task, an LLM decomposes the task into actionable low-level actions.

Acquiring diverse robotic skills with minimal human supervision has garnered considerable attention. However, previous methods have either attempted to chain existing skills, relying heavily on a collection of basic skills [4,8,9], or explored from scratch but often yielded non-interpretable robot behaviors, especially those using unsupervised reinforcement learning [10–13]. We ask whether an LLM can encourage a robot to learn novel tasks that consist of entirely novel yet relevant skills, hypothesizing that incorporating LLMs and textual specifications can guide skill discovery toward more meaningful and human-aligned outcomes. This work takes a step in that direction, though challenges remain in fully closing the gap.

Imagine a robot being placed in a new environment. The robot must be motivated to explore the environment in a way to learn applicable skills before becoming ready to perform further tasks. Given the knowledge about human actions, that resides in LLMs, we expect that an LLM can, by itself, suggest a variety of contextually meaningful skills for the robot to learn. We refer to this autonomous exploration as **A**gentic **S**kill **D**iscovery (ASD), which enables robots to interact with their environment through semantically-driven motivation and autonomously initiate requisite learning procedures. In this research, we address the challenge of open-skill learning in robotic systems guided by LLMs, wherein either LLMs or a complementary vision-language model (VLM)

☆ This article is part of a Special issue entitled: 'Planning & Learning' published in Robotics and Autonomous Systems.

* Corresponding author.
E-mail address: xufeng.zhao@uni-hamburg.de (X. Zhao).

https://doi.org/10.1016/j.robot.2025.105248

Available online 12 November 2025
0921-8890/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

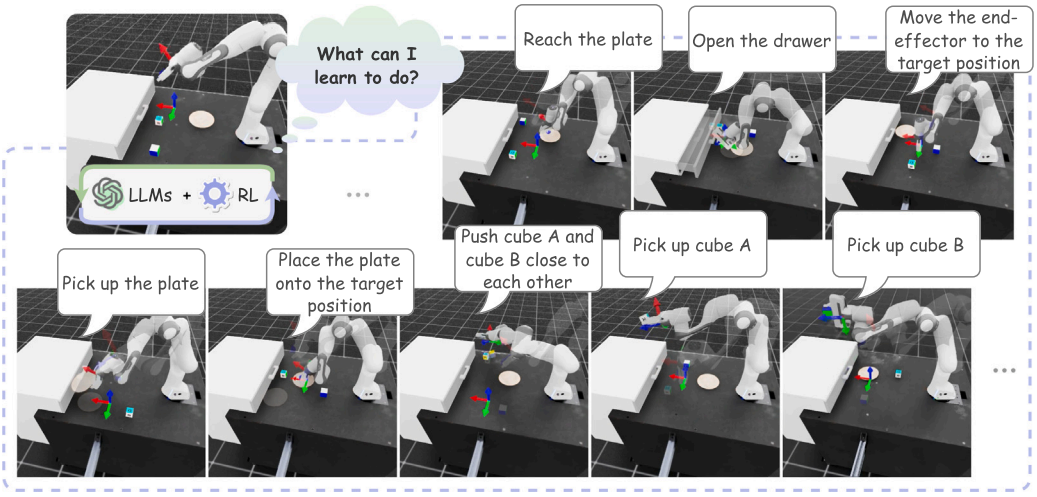


Fig. 1. Guided by LLMs, ASD enables robots to explore the environment and incrementally acquire contextual skills. Tasks proposed by LLMs are learned as skills via parallel reinforcement learning. A high-quality VLM verifies success and ensures reliability.

must facilitate both the learning process and success/failure evaluation without human intervention or predetermined assessment criteria (Fig. 1). Our primary contributions encompass a learning approach that incorporates:

- Task proposal: an LLM iteratively proposes novel and open tasks that are suitable for the given environment (Section 3.1), and collects the resulting skills.
- Task completion for skill learning: existing works such as Eureka [14] focus on solving *determined tasks* where the success/failure is defined by the human in advance, and cannot be generalized to automatic learning of open robot skills. In contrast, we introduce using LLM to propose a success determination function, which provides a criterion for optimizing the reward functions. To make open skill learning tractable, we showcase the importance of having a post-validation VLM for success determination, as a guarantee for reliability and trustworthiness (Section 3.2, Fig. 4) — minimizing false positives and false negatives is critical, so the skill library does not get contaminated with overtrusted skills or depleted from useful skills (Fig. 7). Furthermore, we apply the RAG (Retrieval Augmented Generation) technique to retrieve validated function references for the efficacy of LLM reasoning. We note that the application of RAG to open-skill learning is novel and progressive beyond the state of the art.
- Reuse of skills: to complete certain sophisticated tasks requiring long-horizon manipulations, we prompt LLMs to decompose them into a chain of sub-tasks that are either already explored or are novel to learn on demand, forming a chain of conditional skills.

2. Related work

Skill Discovery. Acquiring diverse robotic skills with minimal or no human supervision is a key challenge in reducing human effort. Unsupervised reinforcement learning (URL) has emerged as a prominent research area for generating diverse behavioral trajectories that are distinguishable from one another [10–13,15]. Typically, URL methods employ information-theoretic objectives during unsupervised training to maximize behavioral entropy. However, this "bottom-up" scheme (i.e., cluttering various trajectories as skills) often results in non-interpretable and semantically meaningless skills from a human perspective, making the learned skills hard to collect and reuse. The trade-off between low costs and meaningful resulting behavior depends on the amount of human knowledge introduced — more supervision (high cost) generally indicates more natural robotic behaviors, and vice versa. While some works introduce further constraints relating to human demonstrations to acquire natural behaviors [15], the scalability is still limited by the need for domain-specific data collection. Recent work LGSD [16] targets the discovery of policy options under a human-specified, constrained subspace (e.g., language-defined goals or curated skill sets). Our formulation seeks fully autonomous skill acquisition without predefined reward structures (e.g., a learning objective using information gain), our focus is on discovering skills at a higher level of abstraction, i.e., in a "top-down" scheme where the learning objectives are ensured to be semantically meaningful in advance, producing a growing repertoire of semantically distinct skills that demonstrate greater diversity and interpretability.

Chaining known skills into new ones can substantially extend robots' abilities and efficiencies. In order to efficiently combine basic skills in a meaningful way and avoid combinatorial explosion, LLMs can be utilized to reason the logical ways of stacking skills to complete new, long-horizon tasks [4,9]. When LLMs are prompted with environment contexts, such as available robot joints and object types, they can propose meaningful motivations for the next movement [4,8]. Prompting an LLM to get an "interesting outcome" can yield generally meaningful skill combinations [9]. However, prior works primarily focus on combining pre-acquired skills to create more complex behaviors. In such approaches, the so-called "new skills" are assembled from an existing library of basic skills rather than developed from scratch. Consequently, the potential skill space is constrained by the foundation of the initial skill set. For example, a robot proficient only in "pushing" would be unable to acquire the skill of "grasping" within the confines of skill assembly. In contrast, the concept of agentic environmental skill discovery, where both the expansion of high-level skill libraries and the initiation of necessary low-level skill training occur autonomously, remains underexplored. To address this gap, our work employs RL agents, supervised by LLMs and VLMs, to acquire novel low-level policies that were previously beyond the robot's capabilities.

Code LLM Control. Most LLM-based robot behavior relies on predefined primitives (skills). This inflexible design makes it difficult to generalize to unseen objects and instructions [2,3]. Recent approaches let code-LLMs write programming code to complete open instructions [17,18]. In particular, VoxPoser [18] uses VLM and LLM to construct a 3D cost map to guide a robot engaging with its surroundings. VoxPoser relies heavily on the quality of the initially composed cost map, limiting its ability to perform exploratory behaviors. In addition, it depends on a trajectory solver during inference, which can further constrain its flexibility and scalability. In contrast, ASD launches reinforcement learning, letting the agent explore the environment and

exploit the learned language conditioned policies. As for the automatic learning of low-level control policies, previous approaches show that LLMs are capable of programming reward functions to optimize over, achieving remarkable performance even for complex tasks [14,19].

However, methods such as Eureka [14] have not been applied to open-skill robot learning, where tasks are newly proposed rather than pre-specified. Unlike Eureka's fixed-task regime with known fitness signals, our setting provides only a language instruction, without predefined success or failure conditions, making learning and evaluation substantially more challenging. Our method partially follows Eureka in using an LLM to propose executable reward functions for RL. However, we differ in three design choices: (1) Skill-Learning (Alg. 2) builds a policy library with multiple options per task and supports long-horizon composition during execution (Alg. 3); (2) decoupled verification, a fast LLM success function used during training and an independent slow VLM verifier for adjudication, mitigates single-model bias; (3) retrieval-augmented prompting over previously learned policies/failures reduces LLM calls and steers reward drafts toward workable structures.

3. Agentic skill discovery

The **A**gentic **S**kill **D**iscovery framework (ASD) introduces a fundamentally different approach to robotic skill acquisition, where skills emerge purely from the interaction between a language model, a robot, and its environment, without any pre-structured human guidance such as demonstrations, reward designs, preferences, or handcrafted supervision signals. At a high level, ASD enables autonomous exploration through self-generated task instructions (Section 3.1), while at a lower level, it learns to master these discovered tasks as skills through reinforcement learning with self-determined success criteria and reward strategies (Section 3.2). The framework then leverages these acquired skills to tackle long-horizon tasks and further expand its skill repertoire through task decomposition (Section 3.3) and *on-demand skill learning*.[1] Unlike traditional approaches where skill complexity and granularity are carefully engineered, ASD faces the unique challenge of operating in an unconstrained space where the language model must discover appropriate skills, determine their complexity, and establish success criteria without predefined constraints or reward structures. This represents a significant departure from existing methods, as it removes human scaffolding from the skill acquisition process while enabling truly autonomous skill discovery and composition. A pseudo-code of the ASD framework is shown in Alg. 2[2]

3.1. Iterative task proposal and skill collection

Instead of relying on exhaustive human efforts, ASD utilizes LLMs to propose meaningful tasks given the description of a certain scene. Those tasks will be assigned to reinforcement learning agents to learn corresponding language-conditioned policies (see Section 3.2). Fig. 2 overviews skill acquisition by the propose-learn-collect loop.

To provide the LLM with sufficient information about the environment, we provide it with the source code of the observation space [14]. Also, the robot configuration, such as robotic arm type and DoFs, is prompted as the initial background description. Due to environmental complexity and unpredictable learning challenges, we implement iterative task proposal and learning rather than allowing the LLM to propose all tasks at once. In particular, the LLM will be informed about tasks that could not be completed so that it will have a sense of the limits of the learning agent, influencing the successive task proposals. For the sake of efficiency and reusability, we encourage the LLM to propose tasks that are meaningful, atomic, independent, and incremental (see Fig. D.11 in Appendix D for detailed prompts).

Skill Options. Generally, control with various *options* for a given task has the potential to be more robust and generalizable [12,20]. As will be introduced in Section 3.2, in the process of the evolutionary search of diverse reward functions, a given task will be successfully learned by several options, forming a set of various control policies. A task will be considered completely learned, or in short, complete, if the resultant agent behavior aligns with expectations (detailed later in 3.2 for skill passing both *fast* and *slow* success determination); otherwise, it is deemed unsuccessful after an extended period of learning.

Completed tasks will be considered as "skills", along with their "options", to be stored in the skill library, and the names of attempted but uncompleted tasks will be added to a failure pool. A summary of the completion status will be generated to guide LLMs in proposing subsequent tasks, taking into account the learning curve and potential difficulties. Failed tasks are usually too sophisticated for LLMs to write reward functions to master at once. Hence, we will decompose and complete them by combining acquired and on-demand-learned skills (Section 3.3). We collect all options for the same skill, i.e. various policies paired with successful reward functions, as future execution candidates, and we leave the study of mixing various skill options for one robust skill control for future research.

3.2. Evolutionary skill learning with fast and slow success determination

LLMs are capable of composing reward functions for RL agents to accomplish specified tasks [14,19]. In our framework, we extend the strategy introduced in Eureka [14], where LLMs are prompted to write reward functions and refine them through evolution based on performance. Specifically, we convert the environment and robot configurations (Python source code) into structured prompts, along with the target task description and a template for the reward function. The LLM generates candidate reward functions in code format, from which we extract valid expressions using regular expressions. These candidate reward functions are then evaluated by training an RL agent; their effectiveness is quantified by success rates, which serve as a fitness score. We retain and mutate only the most successful reward functions over multiple rounds of this evolutionary loop. This iterative strategy enables efficient sampling in pursuit of the discovery of meaningful and learnable reward definitions. See Fig. 3 for an overview of this process and Alg. 1 for pseudo code for details.

Challenges in Open-Skill Learning. In open-skill learning, where the focus is on optimizing newly proposed tasks rather than predefined ones, naive Eureka-like methods [14] cannot be directly applied due to the fact that the success criteria cannot be predetermined. This is because the fitness function, which serves as the ground truth for determining success, is unknown for each task proposal. As a result, evolution selection becomes impossible due to the absence of a unified golden metric to quantify performance. Traditionally in RL, the success determination function for a specific task is programmed by humans into a function that is called at every physics step. In addition to generating reward functions, we also leverage LLMs to compose success functions, which return a binary signal indicating whether a task has been completed. These functions are generated in a similar way: given the environment and task description, the LLM is prompted to write a Python function that evaluates task completion. Compared to reward functions, success functions are inherently simpler, as they do not require shaping dense feedback. In practice, however, it is difficult to exhaustively verify the correctness of all LLM-generated success functions. Instead, we can evaluate their reliability indirectly by comparing the number of final positive policies, i.e., those judged successful by the success function, to the number of true positives, which are validated through manual inspection. This difference provides an approximate measure of success function accuracy. Nevertheless, the soundness of the success function requires investigation. Given that the LLM serves dual roles as both a "player" (for reward function optimization) and as a "referee" (for success determination), employing the resultant

[1] For example, a new "placing" skill should be learned *on demand* when instructed to stack two cubes together, if the skill library contains only primary skills like "pushing" and "picking".

[2] For more implementation details, refer to supplementary materials and open-sourced code at: https://github.com/xf-zhao/Agentic-Skill-Discovery.

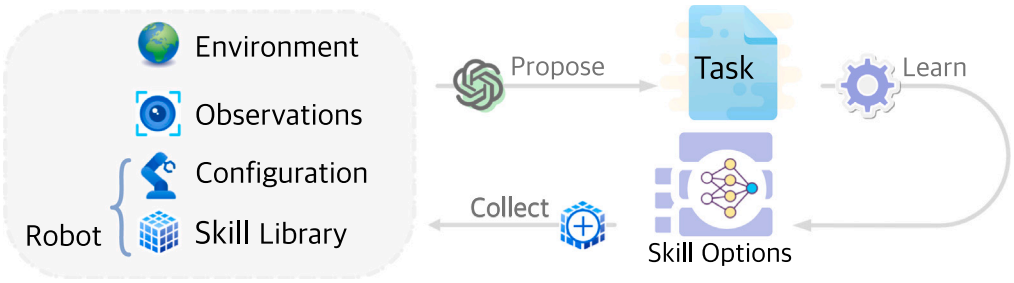


Fig. 2. Contextual skill acquisition loop of ASD. Given the environment setup and the robot's current abilities, an LLM continually proposes tasks for the robot to learn (see Fig. 4 for the learning scheme). Successfully completed skills are collected as acquired skills in the skill library. The learning process for each target skill may yield multiple viable execution strategies, which we preserve as distinct skill options. For each option, we store both the learning specifications (including reward and success functions) and the corresponding trained policy networks, enabling efficient retrieval and deployment of these skills in future tasks.

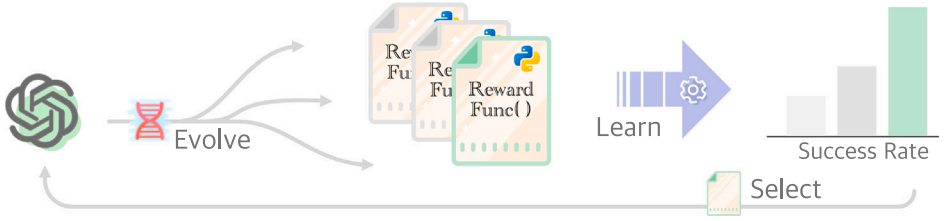


Fig. 3. Evolutionary search of reward functions for *defined tasks with deterministic success functions*, where the success rate can be reliably computed and used as a fitness measure. However, this approach is too simple for novel skill learning, where the absence of prespecified success criteria necessitates a behavior verification mechanism (cf. Fig. 4).

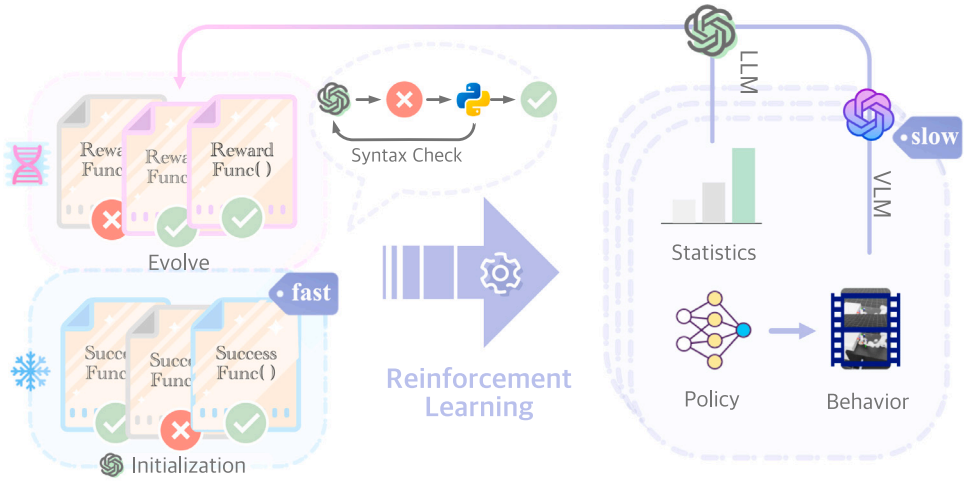


Fig. 4. The evolutionary skill learning procedure of ASD. An LLM composes a set of reward and success functions (*left*), corresponding reinforcement learning to train policies (*middle*), and evolutionary search with learning statistics (e.g., success rate) and VLM assessment (*right*).

success rate as the fitness function for evolution may jeopardize learning stability and trustworthiness. See also Fig. 7 and Section 4.2 for further discussion of the undesired behaviors stemming from evolution with incorrect fitness measurement. The success function and reward function form a chicken-egg relation in that (1) the reward function search relies on reliable success determination and, meanwhile, (2) it is unfeasible to verify the success function before the learning. As a result, it is challenging to have an evolutionary search for both at the same time.

Success Determination: Fast and Slow. Distinguishing whether specific behaviors fulfill a task at each RL step, referred to as *fast success determination* (system 1), as opposed to assessing after fixed intervals of execution, as *slow success determination* (system 2), is pivotal in reinforcement learning. Employing fast success determination enables the agent to receive sparse rewards in real-time and terminate actions promptly to prevent potentially adverse explorations. Although the success function generated by LLMs is essential for RL training, it may also be untrustworthy. Therefore, an independent post-training evaluation is required as a complementary measure to ensure reliability. For instance, the human examination of learned RL behaviors, especially from a draft success function to debug, can be regarded as a slow success determination. Intuitively, VLMs can be applied to analyze robot behaviors. For example, REFLECT [21] uses a multimodal structure to explain execution anomaly. To establish a stable learning cycle, we propose integrating both *fast* and *slow* success evaluations, distinguished by the temporal extent of the underlying processes, to enhance reliability. A rollout is labeled *positive* (P) if it passes a given stage's verification and *negative* (N) otherwise. The four (P/N) combinations across the two stages are summarized in Table 1.

Algorithm 1: POLICY-LEARNING function for reward function generation and policy learning

Input : Environment $\mathcal{E}$; robot $\mathcal{R}$; task $\mathcal{T}$; reward-generation prompt P^{rew}; fast success function f^{succ}; evolution steps K.

Output: Set of survivor policies Π.

$\Pi \leftarrow \emptyset$; $f^{\text{rew}}_{\text{best}} \leftarrow$ `null`; `// best-so-far reward function for prompt carry-over`

for $k \leftarrow 1$ **to** K **do** `// outer evolution steps`

  $F^{\text{rew}} \leftarrow \{ f^{\text{rew}} \leftarrow \text{LLM}(\mathcal{T} \mid P^{\text{rew}} \oplus f^{\text{rew}}_{\text{best}}, \mathcal{E}, \mathcal{R}) \mid$ `SyntaxCheck`$(f^{\text{rew}}) =$ `true` $\}_{m=1}^{N_{\text{rew}}}$ `// generate` N_{rew} `mutated reward candidates conditioned on the current best one`

  bestScore $\leftarrow -\infty$

  foreach $f^{\text{rew}}_m \in F^{\text{rew}}$ **do** `// parallelizable across` f^{rew}_m

    $\pi_m \leftarrow$ `RL-Train`$(\mathcal{T}, f^{\text{succ}}, f^{\text{rew}}_m \mid \mathcal{E}, \mathcal{R})$

    score$_m \leftarrow f^{\text{succ}}(\pi_m)$

    if *score*$_m > 0$ **then** `// collect survivors`

      $\Pi \leftarrow \Pi \cup \{\pi_m\}$

    end

    if *score*$_m >$ *bestScore* **then** `// elitism for next prompt`

      bestScore $\leftarrow$ score$_m$; $f^{\text{rew}}_{\text{best}} \leftarrow f^{\text{rew}}_m$

    end

  end

end

return Π

Table 1
Possible combinations of predictions by fast and slow determination in terms of positive (P) and negative (N) outcomes.

	P_1P_2	P_1N_2	N_1P_2	N_1N_2
Fast (system 1)	✓	✓	✗	✗
Slow (system 2)	✓	✗	✓	✗

fast: sample a set of success functions that are used unchanged throughout skill learning, based on which an LLM launches RL training and deterministically selects reward function survivors. The underlying hypothesis is that determining success is much more attainable than programming an applicable reward function. Policies that pass the fast success determination are considered as *system-1 positives*, P_1, and are collected as survivor candidates.

slow: prompt an independent VLM to additionally examine the success of survivor candidates, before passing them on to the next evolutionary generation. In particular, with the context of the environment and task, a VLM is tasked to describe and assess the reinforced robot behaviors to filter out the ones that also pass the slow determination, i.e. *double positives* $P = P_1P_2$, securing a robust learning loop without human supervision. Since the amount of unsuccessful behaviors during learning (system-1 negatives) is overwhelming to evaluate, and N_1P_2 samples are less common and less harmful,[3] we do not additionally examine them due to the high assessment cost.

[3] N_1P_2 appear only when reward functions are better composed than the success functions, being *less common* according to the hypothesis above. Besides, failures will not contaminate the existing skill library, being *less harmful* than P_1N_2 regarding possible future executions.

Early Misconduct Check. In practice, LLMs may generate unacceptable function designs, e.g. trying to import unsupported third-party Python modules or producing nonsensical outputs (see Appendix C.3 for an example). Some of the potential bugs can be bypassed by carefully designed prompts, while others should be examined at runtime by a Python interpreter. Instead of directly launching RL and feeding back LLMs all kinds of execution errors at the end as in Eureka [14], we carry out early syntax checks and loop until the function generations meet certain requirements. This separate check reduces unnecessary waiting time for simulation preparations and provides an efficient reward search that focuses only on performance feedback. See Fig. 4 for an overview of the evolutionary skill-learning procedure. The corresponding prompts can be found in Appendix D.

Retrieving Skill Specification with RAG. The process of providing LLMs with environmental knowledge necessitates abstraction, which inherently results in information loss. For instance, certain environmental parameters, such as the operational constraints of a drawer moving 10 centimeters along the X-axis, are typically embedded within 3D asset properties rather than explicit scene establishment code. Certain aspects of the environment can only be understood through direct interaction. As a result, learning configurations and their associated outcomes from previous trials provide valuable insights for subsequent experiments. The collection of historical data represents an environmental context distillation process, drawing from both LLMs' environmental awareness and VLMs' behavioral assessment capabilities, being able to gradually compensate losses introduced through abstraction. Despite lacking a dedicated 3D structure interpretation module, our agent can derive aspects of such environmental constraints through iterative experimentation with various learning specifications. This discovered information is subsequently encoded within learning parameters — for example, a selected reward function that incentivizes positive x-coordinate displacement of the drawer handle effectively facilitates successful drawer-opening task completion.

To reduce redundant exploration and effectively integrate prior knowledge, we implement a knowledge accumulation strategy that enables the LLM to retrieve specifications of previously learned skills relevant to the current learning objective. Retrieval-Augmented Generation (RAG) [22], which enhances LLM performance by incorporating contextually relevant information from a local data pool into prompts, has been widely adopted. In our skill learning framework, where success and reward functions are identified through evolutionary search, and where many skills within the same environment share common structural patterns, we apply RAG to enhance LLM prompting. Specifically, we retrieve previously verified skill specifications, including their associated success and reward functions, from the evolving skill library and use them to augment the prompt for generating new functions. This process, which we refer to as skill-RAG, provides the LLM with concrete, context-relevant examples that guide its output and reduce ambiguity. By narrowing the evolutionary search space, skill-RAG improves both the efficiency and reliability of open-ended robot skill acquisition.

3.3. On-demand skill learning with quest decomposition

ASD initially learns skills starting from similar environmental reset states $s_0 \sim \rho_0$, where ρ_0 indicates the initial state distribution with limited randomness, such as object placement. Consequently, some LLM-suggested skills cannot be trained if the pre-conditions are not satisfied. For example, the skill "`placing an object`" requires the initial state of having the object picked. An intuitive solution is to configure the learning environment open-ended/reset-free [23,24], where an LLM continually observes the changes and proposes tasks to complete. However, it challenges both the dynamic sensing ability of LLM as well as RL in practice, especially when RL is accelerated by learning in many parallel environments. Another way is to reset the environment to the final state of executed skills, thereby exploring sequentially

Algorithm 2: SKILL-LEARNING function to accumulate skill library and capability context.

Input : Environment context $\mathcal{E}$; robot configurations $\mathcal{R}$; task proposal prompt P^{task}; prompt P^{succ} for success function generation; task learning iterations N_T; VLM robot behavior success rate threshold ϵ;
Output: Skill library Z (hashmap from task $\mathcal{T}$ to validated options $\Pi^{P_1P_2}$), capability descriptions C.

Initialize $Z \leftarrow \{\}, C \leftarrow null$;
for $i = 1$ **to** N_T **do**
  // Initialize per-task positive sets (fast-only vs. fast+slow)
  $\Pi_i^{P_1} \leftarrow \emptyset, \Pi_i^{P_1P_2} \leftarrow \emptyset$;
  $\mathcal{T}_i \leftarrow \text{LLM}\,(C|\ P^{\text{task}}, \mathcal{E}, \mathcal{R})$; // Contextual task proposal
  // Generate success functions f^{succ} for a given task
  $F^{\text{succ}} \leftarrow \{\, f^{\text{succ}} \leftarrow \text{LLM}(\mathcal{T}_i \mid P^{\text{succ}}, \mathcal{E}, \mathcal{R}) \mid \text{SyntaxCheck}(f^{\text{succ}}) = \text{true} \,\}_{m=1}^{N_{\text{succ}}}$
  foreach $f_j^{succ} \in F^{succ}$ **do**
    $\Pi_{i,j} \leftarrow \text{Policy-Learning}(\mathcal{T}_i, f_j^{\text{succ}} \mid \mathcal{E}, \mathcal{R})$;
    // Accumulate fast-positives (passed f_j^{succ})
    $\Pi_i^{P_1} \leftarrow \Pi_i^{P_1} \cup \Pi_{i,j}$;
    // Slow determination (VLM) per policy option
    $\Pi_i^{P_1P_2} \leftarrow \Pi_i^{P_1P_2} \cup \{\pi | \pi \in \Pi_{i,j}, \text{VLM}(\mathcal{T}_i, \pi | \mathcal{E}, \mathcal{R}) > \epsilon\}$;
  // Collect learning ingredients as capability context
  $C_i \leftarrow \oplus(\mathcal{T}_i, f_i^{\text{succ}*}, f_i^{\text{rew}*}, \Pi_i^{P_1P_2} \neq \emptyset?)$;
  $C \leftarrow C \oplus C_i$;
  $Z[\mathcal{T}_i] \leftarrow \Pi_i^{P_1P_2}$;
return Z, C;

arranged further skills conditioned on already collected skills. Since the bottom-up bootstrapping of skills leads to an explosion of possibilities, we introduce a top-down on-demand learning strategy for complex tasks that RL fails to learn, which we term *quests*. Given a quest selected from the failure pool generated during the skill discovery phase, an LLM is tasked with decomposing it into a sequence of subtasks, thereby establishing a hierarchical RL framework. This decompose-and-conquer strategy has been successfully verified to work [2,3,7].

However, these approaches only allow a decomposition into a limited set of subtasks that can be completed with known skills (bottom-up for completion). In contrast, our method, as illustrated in Fig. 5, allows the LLM to come up with novel and contextually appropriate skills to be learned on demand. This capability enhances the framework's flexibility and generalizability in addressing challenging tasks that require the LLM to construct sophisticated reward functions.

To avoid the forgetting problem in multitasking policy learning, we learn individual policy networks for each RL launch, which constitute the skill options, and only keep the surviving ones according to their performance in the evolution loop. As for learning of the ith on-demand skill, since its initial state is reset as the final state of the last stacked skill π^{i-1}, i.e., $s_0^i = s_T^{i-1} \sim \rho_{\pi^{i-1}}$, we initialize the policy weights from the last learned skill rather than randomly, $\pi^i \leftarrow \pi^{i-1}$. This maintains s_0^i at the beginning of learning and ensures a smooth transition from there when being gradually optimized. Alg. 3 summarizes the on-demand skill learning process.

Algorithm 3: SKILL-RETRIEVING function for quest completion

Input : Quest $\mathcal{Q}$, prompts for quest decomposition P^{decmp}, Robot $\mathcal{R}$, Environment $\mathcal{E}$ with initial state s_0, policy learning function given (initialized as) state s: POLICY-LEARNING$(\cdot|s)$, collected skill library Z_0
Output: Quest Completion

Initialize state $s \leftarrow s_0$;
$\{\mathcal{T}_i\}_{N_T} \leftarrow \text{LLM}\,(\mathcal{R}, \mathcal{E}, P^{\text{decmp}}, \mathcal{Q})$; // Quest decomposition
for $\mathcal{T} \in \{\mathcal{T}_i\}_{N_T}$ **do**
  $k^* \leftarrow \text{argmax}_{k \in \text{Keys}(Z_0)}\, \text{Sim}(\mathcal{T}, k)$;
  if $k^* \neq null$ **and** $\text{Sim}(\mathcal{T}, k^*) \geq \tau$ **and** $Z_0[k^*] \neq \emptyset$ **then**
    // Retrieving (top-1 match) related skill
    $\Pi \leftarrow Z_0[k^*]$;
  else // On-demand skill learning
    $\Pi \leftarrow \text{Policy-Learning}(\mathcal{T}|s)$;
  $\pi^*, s' = \text{argmax}_{\pi \in \Pi, s' \leftarrow \pi(s)}\, \text{VLM}(\mathcal{T}, \pi | \mathcal{E}, \mathcal{R})$;
  $s \leftarrow s'$;

4. Experiments and results

Environment. We run the simulation in the Isaac Sim simulator[4] since it supports parallel environment simulation, which dramatically accelerates the trials of RL with various reward functions. We set up a table scenario with Franka Emika Panda robotic arm, which has 7 DoF and a two-finger gripper. On the table, several objects are randomly placed in front of the arm, with a drawer that can be opened. It is a single scenario, but it enables multiple tasks. See Fig. 6 for the environment setup, with all assets[5] available in Isaac Sim. The experiments aim to answer the following questions:

Q1. What kind of tasks will be proposed? (Section 4.1)
Q2. Can skills be acquired automatically? (Section 4.2)
Q3. How do RL and RAG influence learning? (Section 4.3)
Q4. Can challenging tasks be completed by chaining learned skills? (Section 4.4)

LLMs and VLMs. We employ `gpt-3.5-turbo` to propose tasks and generate reward and fast success functions since it shows a good programming ability while being acceptable regarding cost. For the slow assessment of behaviors generated by policies that have already been positively evaluated using fast success functions, we record the resulting behaviors as videos. Keyframes are then extracted from these videos and analyzed using the advanced VLM GPT-4V(ision),[6] which verifies the successful completion of the tasks.

Learning. With a temperature of 1.0, the LLM samples three success functions in each iteration; based on each, three reward functions are further sampled to launch RL training and to evolve. We set the number of generations of evolutionary search to three. For RL training, we use coordinate states of objects as the state input, and the action space is set to be the robot joint position space. Using joint coordinates instead of Euclidean coordinates avoids control lag caused by inverse kinematics, which would slow down parallel environments. For optimization, we apply the RL framework `rsl_rl`[7] implemented by Orbit [25], where PPO [26] is applied with the same fixed parameters across all potential tasks. For each RL iteration, we configure the maximum permissible

[4] https://docs.omniverse.nvidia.com/isaacsim
[5] https://docs.omniverse.nvidia.com/isaacsim/latest/common/NVIDIA_Omniverse_License_Agreement.html license.
[6] https://openai.com/research/gpt-4v-system-card.
[7] https://github.com/leggedrobotics/rsl_rl

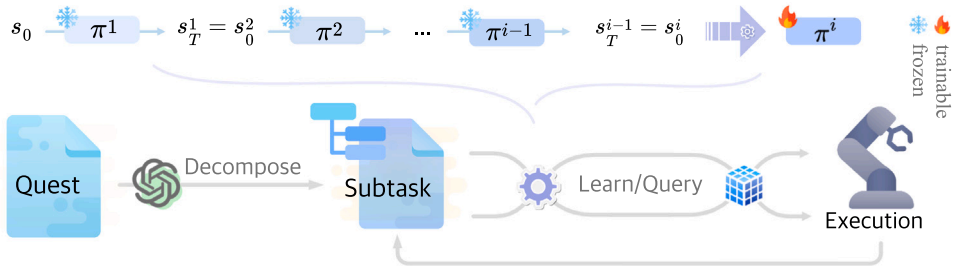


Fig. 5. *Top*: By chaining together learned skills, ASD can further learn a new skill π^i on demand. This is needed when a complex task is too challenging for RL to learn as a whole. *Bottom:* ASD solves quests, namely challenging tasks, with top-down decomposition and skill learning, where the skill library expands for each subtask's completion.

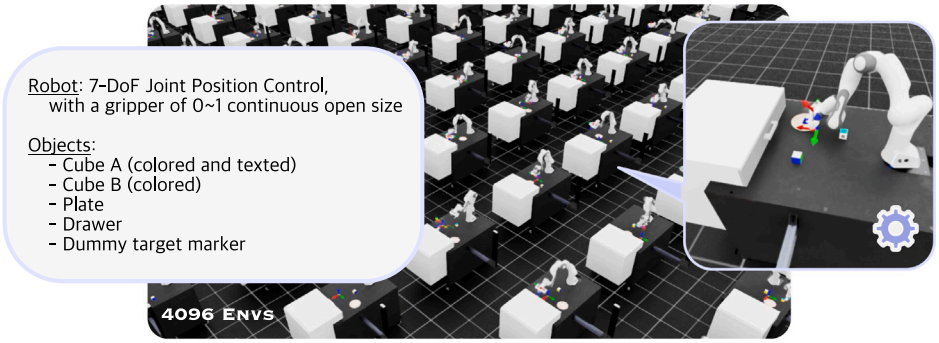


Fig. 6. The simulated scenario setup in Isaac Sim with parallel environments for RL training.

physics steps to 250, with 4096 parallel environments, and a total learning duration of 2000 episodes. We train ASD on 6 NVIDIA GeForce GTX 1080 Ti GPUs, where each proposed task takes around 6 h.

Baseline and Ablation. Given the unpredictable nature of ASD tasks, understanding the learning capabilities of the policy backend is crucial. The ability of code-based LLMs to complete novel tasks has garnered significant research interest [17,18,24]. In ASD, agents are trained to accomplish specified tasks with Eureka-like RL [14]. To evaluate our approach, we use VoxPoser [18] as a comparative baseline for new skill acquisition.

Metrics. To assess the effectiveness of the two system verification and determine whether the proposed tasks have been successfully mastered, we conduct a manual inspection of the behaviors exhibited by the learned skills: TP, the number of human-validated double positives; FP, the number of false positives, i.e. policy candidates pass both verification but indeed fails to complete the task; TN and FN ($N = P_1 N_2$ since candidates are at least P_1 to pass to VLM), the number of policy negatives that are correctly or incorrectly rejected by system 2, respectively. Moreover, to demonstrate the effectiveness of the skill-RAG technique, we conducted additional experiments where skill learning efficacy is approximately measured as the minimum number of GPT calls required to acquire the first TP policy. The efficiency gains of skill-RAG are thus measured through a reduction ratio metric, $g = 1 - N_{\text{GPT}}^{\text{RAG}} / N_{\text{GPT}}^{\text{baseline}}$, where $N_{\text{GPT}}^{\text{RAG}}$ and $N_{\text{GPT}}^{\text{baseline}}$ denote the number of GPT calls required to learn the first TP policy with and without skill-RAG, respectively. The gain $g \leq 1$ and is unbounded below in principle (when $N_{\text{GPT}}^{\text{RAG}}$ is sufficiently larger than $N_{\text{GPT}}^{\text{baseline}}$), and a positive value (when $N_{\text{GPT}}^{\text{RAG}}$ is sufficiently smaller than $N_{\text{GPT}}^{\text{baseline}}$) indicates that skill-RAG is more efficient than the baseline. Table 2 shows the learning outcomes of all proposed tasks.

4.1. Q1. What kind of tasks will be proposed?

Given the robotic table manipulation scenario, the LLM could potentially propose numerous possible tasks. We stop exploring further task proposals after reaching a number of 24. More skill-learning details and statistical reports can be found in Tab. C.6 in Appendix C.4. The first column of Table 2 presents the proposed tasks in sequential order. From this, we derive the following preliminary observations:

- The instructional complexity of tasks increases with successive iterations of the proposal, but the associated learning challenges do not necessarily align with this progression.
- Most of the proposed tasks are meaningful and completable under the setup. Some of the tasks are not appropriately proposed due to LLM's misconception of the initial environment setup (e.g., No. 10 for a wrongly deemed initial state, since the drawer is always initialized as closed, it is reasonable to learn to close the drawer) or of the robot's capabilities (No. 19 for difficulties in a guarantee of "without grasping" requirement).
- By design, all of the tasks are parameter-free language instructions. Hence, the LLM interprets "`reach cube A`" and "`reach cube B`" as distinct skills. However, rather than exhaustively enumerating all possible variable permutations (e.g., A ↔ B), the LLM employs a strategy that prioritizes semantic diversity in skill proposition.

As can be observed from the task proposal list, the tasks are generally atomic and meaningful, but there is still potential for improvement, especially in the understanding of the given initial status of the environment. Minimal human effort to examine non-learnable tasks becomes necessary in this case. In this work, we prompt the coding LLM only with text. Future research involving mixed modalities, e.g., visual observations or even point clouds [27], promises to alleviate this phenomenon.

4.2. Q2. Can skills be acquired automatically?

For each task proposal, there are `3 success functions x 3 runs = 9 possible combinations`, within each run the reward function sampling number and evolutionary iterations are both 3. We have the following observations:

Table 2
In the context of table manipulation scenarios, the learning outcomes encompass all positives identified by system-1. The system-2 validation is subsequently applied, and the resulting outcomes are further scrutinized by humans, focusing on metrics such as $TP_{(2)}$, $FP_{(2)}$, $TN_{(2)}$, and $FN_{(2)}$ (cf. Table 1). Notably, all samples collected are P_1, with specific attention directed towards P_2 and N_2 to explore whether VLM within an autonomous system can effectively serve as a reliable proxy for human evaluation. The column with RAG shows the minimal GPT calls (for one skill option acquisition) reduction ratio (the bigger, the better) of having skill-RAG over previously without skill-RAG. The skill mastering result of RL and a baseline skill learning strategy VoxPoser (Vox.) are marked as ✓(success, with ≥ 90% success rate) or ✗(failure). The symbol "-" indicates inappropriately proposed tasks within the environmental context.

	Task Description	$^{TP}/_{FP}$	$^{TN}/_{FN}$	$g\uparrow$	RL	Vox.
1	`Reach cube A`	4/0	0/2	0.00	✓	✓
2	`Reach cube B`	8/0	0/1	0.79	✓	✓
3	`Reach the plate`	7/0	0/2	0.78	✓	✓
4	`Pick up the cube A`	4/1	3/1	0.02	✓	✗
5	`Pick up the cube B`	2/0	4/0	0.70	✓	✗
6	`Slide cube A from its current position to a target position on the table`	3/0	6/0	−0.20	✓	✗
7	`Open the drawer`	1/1	7/0	0.12	✓	✗
8	`Pick up the plate`	3/0	0/0	0.34	✓	✗
9	`Place the plate onto a target position on the table`	3/1	3/1	0.12	✓	✗
10	`Close the drawer`	–	–	–	–	–
11	`Align cube A and cube B to target positions that are apart from each other.`	0/0	8/0	0.00	✗	✗
12	`Close the drawer with cube A inside.`	0/0	3/0	0.00	✗	✗
13	`Gripper open/close toggle`	1/1	4/0	−0.20	✓	✓
14	`Slide cube B to the table edge without toppling it, aiming for a target position near the edge.`	2/0	0/4	0.17	✓	✗
15	`Align end-effector center over the drawer handle without opening or closing the drawer.`	2/0	1/1	0.24	✓	✓
16	`Navigate the gripper to a target pose above cube B without touching it.`	3/1	1/0	0.56	✓	✓
17	`Gently push the drawer to a partially open or closed position.`	1/1	4/0	0.12	✓	✗
18	`Position cube A directly in front of the drawer handle without blocking the drawer from opening.`	0/2	1/0	0.00	✗	✗
19	`Swap positions of cube A and cube B without grasping.`	0/0	3/0	0.00	✗	✗
20	`Move end-effector over cube A.`	7/0	0/2	0.79	✓	✗
21	`Push cube A and cube B close to each other.`	1/0	3/0	0.68	✓	✗
22	`Move to a target position on the table without interacting with objects.`	7/0	1/1	0.82	✓	✓
23	`Put cube A into the drawer.`	0/1	4/0	0.00	✗	✗
24	`Stack cube A on top of cube B.`	0/1	7/0	0.00	✗	✗

O1: Skill options are successfully acquired for most of the proposed tasks. As shown in the TP (true positives) column of Table 2, ASD autonomously acquires a substantial number of valid skill options, 17 out of 24 in total (marked by "✓" for any task with non-zero TP), with the number varying depending on the complexity of the task.

O2: The VLM-based slow success verification effectively filters out false positives from the fast (system-1) evaluation. The TN (true negatives) column indicates that many behaviors were incorrectly judged as successful by the coding LLM, highlighting the need for an additional verification step. Without this, noisy false positives could accumulate due to overreliance on LLM-generated success criteria, potentially leading to erroneous *false learning cycles* (illustrated in Fig. 7). Quantitatively, among all P_1 candidates (i.e., those passing the fast evaluation), only $\frac{TP+FN}{TP+FP+TN+FN} = 46.43\%$ are actually reliable. In contrast, the slow verification mechanism achieves an average precision of $\frac{TP}{TP+FP} = 76.50\%$ with a recall of $\frac{TP}{TP+FN} = 90.74\%$. This significant improvement in precision through VLM-based slow verification reduces the incidence of false positives and thereby enhances both the stability of the learning process and the reliability of the acquired skills.

While our approach leverages LLMs to guide skill specification, we acknowledge that some discovered behaviors may still lack interpretability or task relevance, an issue shared with prior unsupervised skill discovery methods. However, we posit that grounding the skill generation process in natural language prompts and VLM-based verification introduces an interpretable intermediate layer (e.g., textual success and reward functions), which offers more semantic transparency than purely unsupervised objectives. Moreover, human-validate results suggest that many acquired skills align with human-understandable goals. Further ensuring meaningful skill acquisition may require moving beyond costly RL from scratch toward leveraging large pretrained policy models, e.g., vision-language-action (VLA) models, that provide strong priors within the ASD framework for reliable skill discovery adapted to new embodiments and environments, which we leave for future work.

4.3. Q3. How do RL and RAG influence learning?

By alternatively applying VoxPoser [18] as the skill learning strategy, we show the advantages of RL in learning skills from scratch. As shown in Table 2, many tasks can be accomplished using RL but fail when using VoxPoser. This discrepancy can be attributed to VoxPoser's reliance on extensive human effort to predefine spatial hooks and basic motion primitives, coupled with its lack of the exploration capability inherent to RL. Since the goal of agentic skill discovery is to acquire skills with minimal or no human involvement, we do not meticulously design primitives for VoxPoser but instead, provide only basic movement examples. Consequently, this baseline performs

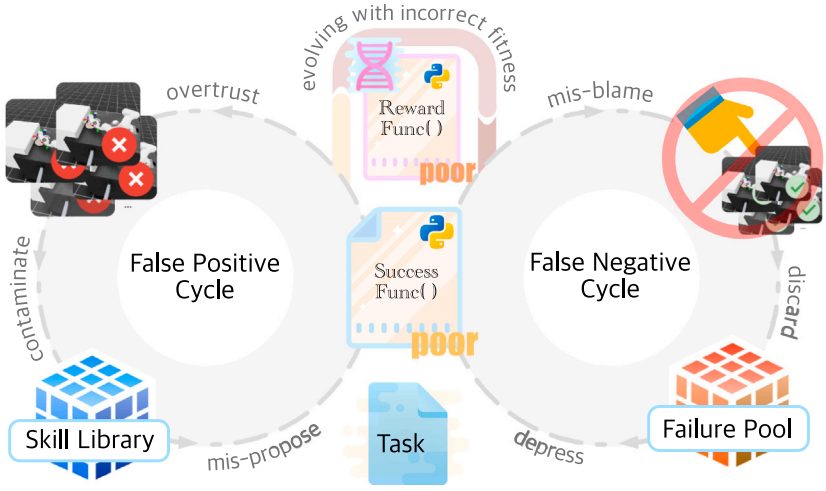


Fig. 7. Possible failure modes with unassured evolution fitness measurement. *Left*: the LLM-generated success determination may confidently but wrongly assess a task as successful, leading to an undesired *false positive learning cycle* with a contaminated skill library and non-accomplishable future task proposals. *Right*: negative evaluation of indeed successful behaviors will misguide the reward function search and prevent skill acquisition, resulting in a *false negative learning cycle*.

well on tasks requiring simple positional approaches but often fails when more complex manipulations are needed. In contrast, RL-based skill learning, driven by its intrinsic exploration capability, successfully adapts to the environment and masters a broader range of skills.

Table 2 demonstrates the advantages of prompting LLMs with previously validated success and reward functions. These retrieved exemplars serve as in-context guidance, enhancing the quality and alignment of newly generated specifications. By leveraging these evolved functions as design cues for the current task, the number of GPT calls required to acquire the first valid skill option is significantly reduced. This efficiency gain is quantified by the reduction ratio g, defined over the number of GPT calls needed to obtain the first true positive (TP) policy. We argue that as the complexity of robotic environments increases, RAG becomes increasingly essential for generating high-quality reward functions. In particular, RAG reduces syntax errors and improves the correctness of low-level tensor operations, e.g., aligning objects vertically, thereby enhancing both the reliability and searching efficiency of agentic skill discovery.

4.4. Q4. Can challenging tasks be completed by chaining skills?

Our empirical analysis reveals that certain proposed tasks necessitate extended execution horizons and present significant challenges for LLMs in formulating effective reward functions. These complex tasks, which we term as a quest in Section 3.3, typically involve multi-step entity manipulations. For instance, a stacking task requires sequential actions: first grasping one object, and then identifying and aligning it with another object. In our observations, even the highest-performing behaviors predominantly focused on various cube-grasping techniques but failed to progress beyond holding the cube in an elevated position. As illustrated in Fig. 5, ASD overcomes this limitation by utilizing LLMs to decompose a quest into sequential short-horizon subtasks. Table 3 showcases two illustrative cases where ASD initially struggled to master complex tasks as unified objectives but succeeded by systematically decomposing them and addressing the subtasks individually. This demonstrates that the policy π^i can be effectively learned and integrated hierarchically by building on existing skills, underscoring ASD's potential for solving long-horizon tasks.

5. Conclusion

Agentic Skill Discovery (ASD) addresses a broad vision for agentic AI systems [28–31], enabling robots to understand complex embodiments and autonomously pursue intricate goals with minimal human intervention. By using LLMs to devise, motivate, and improve necessary learning processes, we have shown that language-conditioned robotic skills can be discovered from scratch, where RL and RAG techniques are beneficial for the efficacy and efficiency of skill learning. Using a VLM for third-party behavior assessment prevents the skill library from being influenced by false positives. Furthermore, ASD also promises to tackle challenging, long-horizon tasks by dividing and conquering on demand and thereby further effectively extending the skills.

Table 3
Snippet of quest completion demonstrating the integration of accumulated skills from the library with on-demand learned skills, where the latter are conditionally acquired using the end states of executed skills as initial states. The symbol "⊳" indicates a replay of already acquired skills in Table 2.

	Quest decomposition	N_{HO}/N_O	N_{HC}/N_C
	`Stack cube A on top of cube B`		
1)	`Pick up cube A`	⊳	⊳
2)	`Place cube A on top of cube B carefully, aligning their surfaces to stack them`	1/2	0/22
	`Put cube A on top of the plate`		
1)	`Pick up cube A`	⊳	⊳
2)	`Place cube A on top of the plate`	3/3	0/3

6. Discussion, limitations, and future work

Abstraction and environment diversity. As discussed in Section 4.1, relying on text-based LLMs to interpret environmental information inevitably introduces a level of context abstraction. This abstraction may obscure fine-grained environmental cues critical for precise behavior learning and evaluation. Moreover, using LLMs to describe and assess robot behavior introduces potential biases and limitations in the evaluation process. A valuable future direction would be to develop and fine-tune specialized robot behavior assessment models [32], leveraging existing multimodal robotic datasets to improve precision and reliability. Additionally, while the current experimental validation focuses on a robotic arm scenario, which effectively demonstrates the framework's capabilities, it remains a relatively constrained task domain. Extending the framework to more complex and diverse scenarios, such as deformable object manipulation, multi-agent coordination, or long-horizon tasks, will be important to further evaluate its generality and robustness.

Simulation and real-world applicability. In addition, the applicability of this method to real-world scenarios and diverse embodiments could be evaluated to fully explore its *agenticness* [28], especially in cases where parallel learning is necessary. Learning-based methods are typically data-intensive, making direct training on real robots slow, resource-exhaustive, and often impractical. To address this, ASD relies on parallel simulation for scalable skill discovery. One emerging direction involves building digital twin environments to safely and efficiently explore policies before transferring them to physical systems. Another strategy is to first collect real-world data to build a simulation model (real-to-sim), then train in simulation, and finally deploy back to the real robot (sim-to-real [33,34]). These real-to-sim-to-real approaches [35,36] are increasingly adopted in recent research to improve transferability and reduce the cost of real-world trials. Moreover, even in the absence of a physical robot, the ASD framework remains useful, e.g., in discovering reusable skills for digital agents, such as virtual character animation or game behavior modeling.

Toward long-horizon autonomy. While the approach shows promising results, it does not guarantee completeness or correctness. Some valid skills may be missed, and others may be incorrect due to limitations in task understanding, verification, or supervision. These challenges reflect the broader difficulty of grounding language in complex environments. We expect such limitations to be reduced through advances in VLMs, large robotic reasoning models, and scalable human-in-the-loop protocols. The current implementation focuses on acquiring basic skills; however, ASD lays a foundation for scalable autonomy. Future research should explore mechanisms for chaining acquired skills sequentially or integrating them into graph-structured representation of policies to enable long-horizon tasks. Robustness is another key factor, i.e. learned skills must remain effective across varying contexts. To this end, methods such as domain randomization and task-level precondition verification should be investigated to improve skill generalization and reliable activation. In addition, extending our option-generation mechanism (options via diverse reward functions) to other alternatives, e.g., an LGSD-style [16] option discovery (via unsupervised RL within a constrained language subspace), will potentially further strengthen skill robustness.

CRediT authorship contribution statement

Xufeng Zhao: Writing – original draft, Software, Resources, Methodology, Formal analysis, Data curation, Conceptualization. **Cornelius Weber:** Writing – review & editing, Methodology, Conceptualization. **Stefan Wermter:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We gratefully acknowledge support from the China Scholarship Council (CSC) and the German Research Foundation (DFG) under the project Crossmodal Learning (TRR 169). We also thank OpenAI and their Researcher Access Program for generously providing GPT-4 API tokens support.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.robot.2025.105248.

Data availability

I have shared the link to codes is available at https://github.com/xf-zhao/Agentic-Skill-Discovery.

References

[1] C. Christodouloupoulos, M. Steedman, A massively parallel corpus: The bible in 100 languages, Lang. Resour. Eval. 49 (2) (2015) 375–395.

[2] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, C. Parada, K. Rao, P. Sermanet, A.T. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, N. Brown, M. Ahn, O. Cortes, N. Sievers, C. Tan, S. Xu, D. Reyes, J. Rettinghouse, J. Quiambao, P. Pastor, L. Luu, K.-H. Lee, Y. Kuang, S. Jesmonth, N.J. Joshi, K. Jeffrey, R.J. Ruano, J. Hsu, K. Gopalakrishnan, B. David, A. Zeng, C.K. Fu, Do as I can, not as I say: Grounding language in robotic affordances, in: K. Liu, D. Kulic, J. Ichnowski (Eds.), Proceedings of the 6th Conference on Robot Learning, in: Proceedings of Machine Learning Research, vol. 205, PMLR, 2023, pp. 287–318.

[3] X. Zhao, M. Li, C. Weber, M.B. Hafez, S. Wermter, Chat with the environment: Interactive multimodal perception using large language models, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2023, pp. 3590–3596.

[4] J. Zhang, J. Zhang, K. Pertsch, Z. Liu, X. Ren, M. Chang, S.-H. Sun, J.J. Lim, Bootstrap your own skills: Learning to solve new tasks with large language model guidance, in: 7th Annual Conference on Robot Learning, 2023, p. 2310.10021.

[5] Z. Mandi, S. Jain, S. Song, Roco: Dialectic multi-robot collaboration with large language models, in: 2024 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2024, pp. 286–299.

[6] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, T. Funkhouser, Tidybot: Personalized robot assistance with large language models, Auton. Robots 47 (8) (2023) 1087–1102.

[7] Y. Ding, X. Zhang, C. Paxton, S. Zhang, Task and motion planning with large language models for object rearrangement, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2023, pp. 2086–2092.

[8] Y. Du, O. Watkins, Z. Wang, C. Colas, T. Darrell, P. Abbeel, A. Gupta, J. Andreas, Guiding pretraining in reinforcement learning with large language models, in: A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, J. Scarlett (Eds.), Proceedings of the 40th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 202, PMLR, 2023, pp. 8657–8677.

[9] B. Celik, A. Ahmetoglu, E. Ugur, E. Oztop, Developmental scaffolding with large language models, in: IEEE International Conference on Development and Learning, 2023.

[10] M. Li, X. Zhao, J.H. Lee, C. Weber, S. Wermter, Internally rewarded reinforcement learning, in: A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, J. Scarlett (Eds.), Proceedings of the 40th International Conference on Machine Learning, ICML, in: Proceedings of Machine Learning Research, vol. 202, PMLR, 2023, pp. 20556–20574.

[11] S. Park, K. Lee, Y. Lee, P. Abbeel, Controllability-aware unsupervised skill discovery, in: A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, J. Scarlett (Eds.), Proceedings of the 40th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 202, PMLR, 2023, pp. 27225–27245.

[12] B. Eysenbach, A. Gupta, J. Ibarz, S. Levine, Diversity is all you need: Learning skills without a reward function, in: International Conference on Learning Representations, 2019.

[13] A. Sharma, S. Gu, S. Levine, V. Kumar, K. Hausman, Dynamics-aware unsupervised discovery of skills, in: International Conference on Learning Representations, 2020.

[14] Y.J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, A. Anandkumar, Eureka: Human-level reward design via coding large language models, in: The Twelfth International Conference on Learning Representations, 2024, p. 2310.12931.

[15] X.B. Peng, Y. Guo, L. Halper, S. Levine, S. Fidler, ASE: Large-scale reusable adversarial skill embeddings for physically simulated characters, ACM Trans. Graph. 41 (4) (2022).

[16] S. Rho, L. Smith, T. Li, S. Levine, X.B. Peng, S. Ha, Language guided skill discovery, in: The Thirteenth International Conference on Learning Representations, 2025.

[17] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, A. Zeng, Code as policies: Language model programs for embodied control, in: Workshop on Language and Robotics At CoRL 2022, 2022, p. 2209.07753.

[18] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, L. Fei-Fei, VoxPoser: Composable 3D value maps for robotic manipulation with language models, in: 7th Annual Conference on Robot Learning, 2023.

[19] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M.G. Arenas, H.-T.L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, F. Xia, Language to rewards for robotic skill synthesis, in: J. Tan, M. Toussaint, K. Darvish (Eds.), Proceedings of the 7th Conference on Robot Learning, in: Proceedings of Machine Learning Research, vol. 229, PMLR, 2023, pp. 374–404.

[20] K. Gregor, D.J. Rezende, D. Wierstra, Variational intrinsic control, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Workshop Track Proceedings, OpenReview.net, 2017.

[21] Z. Liu, A. Bahety, S. Song, REFLECT: Summarizing robot experiences for failure explanation and correction, in: 7th Annual Conference on Robot Learning, 2023, p. 2306.15724.

[22] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, 2024, p. 2312.10997, arXiv:2312.10997.

[23] A. Gupta, J. Yu, T.Z. Zhao, V. Kumar, A. Rovinsky, K. Xu, T. Devlin, S. Levine, Reset-free reinforcement learning via multi-task learning: Learning dexterous manipulation behaviors without human intervention, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, 2021, pp. 6664–6671.

[24] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, Voyager: An open-ended embodied agent with large language models, Trans. Mach. Learn. Res. (2024) 2305.16291.

[25] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J.L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, Orbit: A unified simulation framework for interactive robot learning environments, IEEE Robot. Autom. Lett. 8 (6) (2023) 2301.04195.

[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, p. 1707.06347, CoRR, arXiv:1707.06347.

[27] R. Zhang, J. Han, C. Liu, P. Gao, A. Zhou, X. Hu, S. Yan, P. Lu, H. Li, Y. Qiao, Llama-adapter: Efficient fine-tuning of language models with zero-init attention, in: The Twelfth International Conference on Learning Representations, 2024, p. 2303.16199.

[28] Y. Shavit, S. Agarwal, M. Brundage, S. Adler, C. O'Keefe, R. Campbell, T. Lee, P. Mishkin, T. Eloundou, A. Hickey, K. Slama, L. Ahmad, P. McMillan, A. Beutel, A. Passos, D.G. Robinson, Practices for Governing Agentic AI Systems, Technical report, 2023.

[29] Sequoia Capital, What's next for AI agentic workflows ft. andrew ng of AI fund, 2024.

[30] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, C. Wu, Aflow: Automating agentic workflow generation, in: The Thirteenth International Conference on Learning Representations, 2025, p. 2410.10762.

[31] S. Qiao, R. Fang, Z. Qiu, X. Wang, N. Zhang, Y. Jiang, P. Xie, F. Huang, H. Chen, Benchmarking agentic workflow generation, in: The Thirteenth International Conference on Learning Representations, 2025.

[32] Y.J. Ma, V. Kumar, A. Zhang, O. Bastani, D. Jayaraman, Liv: Language-image representations and rewards for robotic control, in: International Conference on Machine Learning, PMLR, 2023, pp. 23301–23320.

[33] S. Höfer, K. Bekris, A. Handa, J.C. Gamboa, M. Mozifian, F. Golemo, C. Atkeson, D. Fox, K. Goldberg, J. Leonard, C. Karen Liu, J. Peters, S. Song, P. Welinder, M. White, Sim2Real in robotics and automation: Applications and challenges, IEEE Trans. Autom. Sci. Eng. 18 (2) (2021) 398–400.

[34] C. Gäde, M. Kerzel, E. Strahl, S. Wermter, Sim-to-real neural learning with domain randomisation for humanoid robot grasping, in: E. Pimenidis, P. Angelov, C. Jayne, A. Papaleonidas, M. Aydin (Eds.), Artificial Neural Networks and Machine Learning – ICANN 2022, Springer International Publishing, Cham, 2022, pp. 342–354.

[35] X. Li, J. Li, Z. Zhang, R. Zhang, F. Jia, T. Wang, H. Fan, K.-K. Tseng, R. Wang, RoboGSim: A Real2Sim2Real robotic Gaussian splatting simulator, 2024, p. 2411.11839, arXiv:2411.11839.

[36] M.T. Villasevil, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, P. Agrawal, Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation, in: Proceedings of Robotics: Science and Systems, Delft, Netherlands, 2024.